

# Algorithm Cheat Sheet · Which to Use

Find the first matching clue, then take the technique it points to. *Italic phrases under each pill are the words to look for in the problem statement.* Inspired by AlgoMonster's flowchart.

## Graphs & trees

### Tree structure?

Level order or min depth?  
**Yes** **BFS**  
*shortest unweighted path · level order*

Count or build tree shapes?  
**Yes** **Divide & Conquer / Tree DP**  
*count distinct trees · subtree DP*

**No** **DFS**  
*tree paths · explore all branches*

### Directed acyclic graph?

**Yes** **Topological Sort**  
*"course schedule" · prerequisites · DAG*

### Shortest path?

Weighted edges?  
**Yes** **Dijkstra's Algorithm**  
*shortest path · weighted edges*

**No** **BFS**  
*shortest unweighted path · level order*

### Connected components?

**Yes** **Disjoint Set Union**  
*connected components · "merge accounts"*

### Tiny input?

**Yes** **DFS / Backtracking**  
*enumerate paths · small graph*

**No** **BFS**  
*shortest unweighted path · level order*

## Sorted, search & k-th

### Sorted in, monotonic out?

Range/rank queries + updates?  
**Yes** **Ordered Set / Fenwick / Segment Tree**  
*range sum + updates · order stats*

**No** **Binary Search**  
*sorted array · "minimize the max"*

### Top-k or k-th?

**Yes** **Heap / Sorting**  
*kth largest · "top k" · running median*

## Linked lists

### Linked list?

Fast/slow or offset pointers?  
**Yes** **Two pointers**  
*pair from ends · fast/slow · in-place*

Merge several sorted lists?  
**Yes** **Heap / Divide & Conquer**  
*merge k sorted lists*

**No** **Linked-List Manipulation**  
*reverse / reorder a list*

## Lookups, intervals, partition, strings

### Lookup, count, or group?

**Yes** **Hash Table / Counting**  
*frequency · "first unique" · group by*

### Overlapping intervals?

**Yes** **Sorting + Interval Scan**  
*merge intervals · "meeting rooms"*

### Partition in place?

**Yes** **Two pointers / Partitioning**  
*move zeroes · Dutch flag · in-place*

### Match/split via dictionary?

Prefix/pattern search?  
**Yes** **Trie / String Matching / Rolling Hash**  
*prefix search · "starts with"*

**No** **Trie / DP / Memoization**  
*word break · dictionary split*

## Small constraints (n is tiny)

Brute force viable?  
**Yes** **Brute Force / Backtracking**  
*all permutations / subsets ·  $n \leq 12$*

### Track a subset?

**Yes** **Bitmask DP**  
 *$n \leq 20$  · "visit all" · subset state*

**No** **Dynamic Programming**  
*"number of ways" · max/min with choices*

## Subarrays & windows

### Running sums?

**Yes** **Prefix Sums**  
*subarray sum = k · negatives allowed · cumulative totals*

### Contiguous span?

Window invariant?  
**Yes** **Sliding Window**  
*longest/shortest substring ·  $\leq k$  distinct  
exactly k  $\rightarrow$  atMost(k) - atMost(k-1)*

### Nearest greater/smaller?

**Yes** **Monotonic Stack**  
*next greater · "daily temperatures"*

**No** **Dynamic Programming**  
*"number of ways" · max/min with choices*

## Counting arrangements

### Brute force viable?

**Yes** **Brute Force / Backtracking**  
*all permutations / subsets ·  $n \leq 12$*

**No** **Dynamic Programming**  
*"number of ways" · max/min with choices*

### Several sequences?

Monotonic property?  
**Yes** **Two pointers**  
*pair from ends · fast/slow · in-place*

### Overlapping subproblems?

**Yes** **Dynamic Programming**  
*"number of ways" · max/min with choices*

### Locate indices?

Monotonic property?  
**Yes** **Two pointers**  
*pair from ends · fast/slow · in-place*

### Constant space?

Monotonic property?  
**Yes** **Two pointers**  
*pair from ends · fast/slow · in-place*

## Compute a max / min

### Monotonic search space?

**Yes** **Binary Search**  
*sorted array · "minimize the max"*

### Nearest greater/smaller?

**Yes** **Monotonic Stack**  
*next greater · "daily temperatures"*

### Overlapping subproblems?

**Yes** **Dynamic Programming**  
*"number of ways" · max/min with choices*

**No** **Greedy Algorithms**  
*intervals · "jump game" · local choice*

## Parsing, design, simulation, math (the tail)

### Parsing symbols?

#### Count or optimise segments?

**Yes** **Dynamic Programming**  
*"number of ways" · max/min with choices*

**No** **Stack**  
*valid parentheses · expression parse*

#### Design a structure?

**Yes** **Design + Supporting Structures**  
*"implement LRU / iterator"*

#### Just follow the rules?

**Yes** **Simulation / Basic DSA**  
*follow the rules step by step*

### Math or bit tricks?

#### Properties of numbers?

**Yes** **Number Theory**  
*gcd / primes / modular arithmetic*

**No** **Math / Bit Manipulation**  
*XOR tricks · powers · "single number"*

#### Otherwise

**Specialized / Advanced Pattern**  
*rare / problem-specific trick*

# Algorithm Cheat Sheet · How They Work

The mechanism behind each technique on the front — minimal Python-ish templates, not full implementations. Badge = typical time complexity.

## BFS

**O(V+E)**

Explore level by level. Shortest path in an unweighted graph.

```
# adj[u] = node u's neighbour nodes (v)
from collections import deque
q, seen = deque([src]), {src}
while q:
    u = q.popleft()
    for v in adj[u]:
        if v not in seen:
            seen.add(v); q.append(v)
```

## DFS

**O(V+E)**

Go deep, then backtrack. Trees, paths, components, cycles.

```
def dfs(u):
    seen.add(u)
    for v in adj[u]:
        if v not in seen:
            dfs(v)
```

## Topological Sort

**O(V+E)**

Order a DAG so every edge points forward (Kahn's algorithm).

```
from collections import deque
indeg = [0]*n
for u in range(n):
    for v in adj[u]: indeg[v] += 1
q = deque(u for u in range(n) if indeg[u]==0)
order = []
while q:
    u = q.popleft(); order.append(u)
    for v in adj[u]:
        indeg[v] -= 1
        if indeg[v] == 0: q.append(v)
# len(order) < n => graph has a cycle
```

## Dijkstra

**O(E log V)**

Shortest path with non-negative weights, via a min-heap.

```
import heapq
dist = [float('inf')]*n; dist[src] = 0
pq = [(0, src)]
while pq:
    d, u = heapq.heappop(pq)
    if d > dist[u]: continue
    for v, w in adj[u]:
        if d + w < dist[v]:
            dist[v] = d + w
            heapq.heappush(pq, (dist[v], v))
```

## Union-Find (DSU)

**~O(1) amort.**

Merge sets, test connectivity, detect cycles. Path compression.

```
parent = list(range(n))
def find(x):
    while parent[x] != x:
        # path-compress
        parent[x] = parent[parent[x]]
        x = parent[x]
    return x
def union(a, b):
    parent[find(a)] = find(b)
```

## Binary Search

**O(log n)**

Halve a sorted (or monotonic) space. Also "search on the answer".

```
def lower_bound(a, ok): # first True index
    lo, hi = 0, len(a) # on answer: hi=max+1
    while lo < hi:
        mid = (lo + hi) // 2
        if ok(mid): hi = mid
        else: lo = mid + 1
    return lo
```

## Two Pointers

**O(n)**

Walk a sorted array from both ends, or fast/slow on a list.

```
def two_sum(a, target): # a is sorted
    i, j = 0, len(a) - 1
    while i < j:
        s = a[i] + a[j]
        if s == target: return (i, j)
        if s < target: i += 1
        else: j -= 1
```

## Sliding Window

**O(n)**

Grow right, shrink left to keep an invariant (e.g.  $\leq k$  distinct).

```
from collections import Counter
left = 0; cnt = Counter()
for right, x in enumerate(a):
    cnt[x] += 1
    while len(cnt) > k: # shrink
        cnt[a[left]] -= 1
        if cnt[a[left]] == 0: del cnt[a[left]]
        left += 1
    best = max(best, right - left + 1)
# exactly k = atMost(k) - atMost(k-1)
```

## Heap / Top-k

**O(n log k)**

Keep the k best seen so far in a size-k min-heap.

```
import heapq
heap = [] # min-heap, size k
for x in a:
    heapq.heappush(heap, x)
    if len(heap) > k: heapq.heappop(heap)
# heap[0] is the k-th largest
```

## Merge Intervals

**O(n log n)**

Sort by start, then fold each interval into the last if they touch.

```
a.sort()
merged = [a[0]]
for s, e in a[1:]:
    if s <= merged[-1][1]:
        merged[-1][1] = max(merged[-1][1], e)
    else:
        merged.append([s, e])
```

## Dynamic Programming

**O(n·W)**

Build answers from overlapping subproblems (0/1 knapsack shown).

```
def knapsack(items, W):
    dp = [0]*(W + 1)
    for wt, val in items:
        for c in range(W, wt-1, -1): # 0/1: down
            dp[c] = max(dp[c], dp[c-wt] + val)
    return dp[W]
```

## Memoization

**O(states)**

Top-down DP — recurse on subproblems, cache each result in a dict.

```
memo = {}
def fib(n):
    if n <= 1: return n
    if n in memo: return memo[n]
    memo[n] = fib(n-1) + fib(n-2)
    return memo[n]
```

## Backtracking

**O(2<sup>n</sup>) / O(n!)**

Choose, recurse, undo. Permutations & subsets when n is tiny.

```
def backtrack(start, path):
    res.append(path[:]) # record
    for i in range(start, n):
        path.append(a[i])
        backtrack(i + 1, path) # i+1 => subsets
    path.pop() # undo
```

## Monotonic Stack

**O(n)**

Stack holds a sorted run; pop when the new value breaks it.  
Example: next greater value to the right; [2,1,3] → [3,3,-1]

```
stack = []; res = [-1]*n
for i, x in enumerate(a):
    while stack and a[stack[-1]] < x:
        res[stack.pop()] = x
    stack.append(i)
```

## Prefix Sums

**O(n) build**

Precompute cumulative totals for O(1) range-sum queries.

```
prefix = [0]*(n + 1)
for i, x in enumerate(a):
    prefix[i+1] = prefix[i] + x
# sum(l..r) = prefix[r+1] - prefix[l]
# subarray=k: count prefix[] in a hashmap
```

## Trie

**O(len) / op**

Prefix tree for word lookups and "starts-with" queries.

```
def insert(root, word):
    cur = root
    for c in word:
        cur = cur.setdefault(c, {})
    cur['$'] = True # word ends here
```